

A Provenance-Aware Framework for Ensuring Reproducibility in High-Performance Computing

Md Kamal Hossain Chowdhury, Manjila Singh, Hari Teja Jajula, Trupesh Patel, Arpita Patel, Purushotham Bangalore
The University of Alabama, Alabama Water Institute

The Reproducibility Crisis in HPC

- The credibility of scientific research relies on reproducibility.
- In HPC systems, complex software, massive datasets, and extreme parallelism make it hard.
- Lack of reproducibility
 - wastes resources, erodes
 - public trust in scientific findings
 - hinders collaboration.

Key Challenges Addressed

- System Complexity: Hardware and software Variations.
- Lack of Provenance: Difficult to track.
- Data Integrity: Verification is not easy.
- Performance Overhead: resource intensive and slow provenance track.

A Provenance-Aware, Containerized Framework

A novel framework that combines two key technologies:

- Containers (Docker/Singularity)
- Transparent Provenance Collection

How It Works: Key Features

- Automated Research Metadata Collection.
- A Verifiable Workflow:
 - The system is built and run inside a container.
 - Input files are verified via checksums.
 - Execution parameters are stored.
 - Any changes to output files are traced.

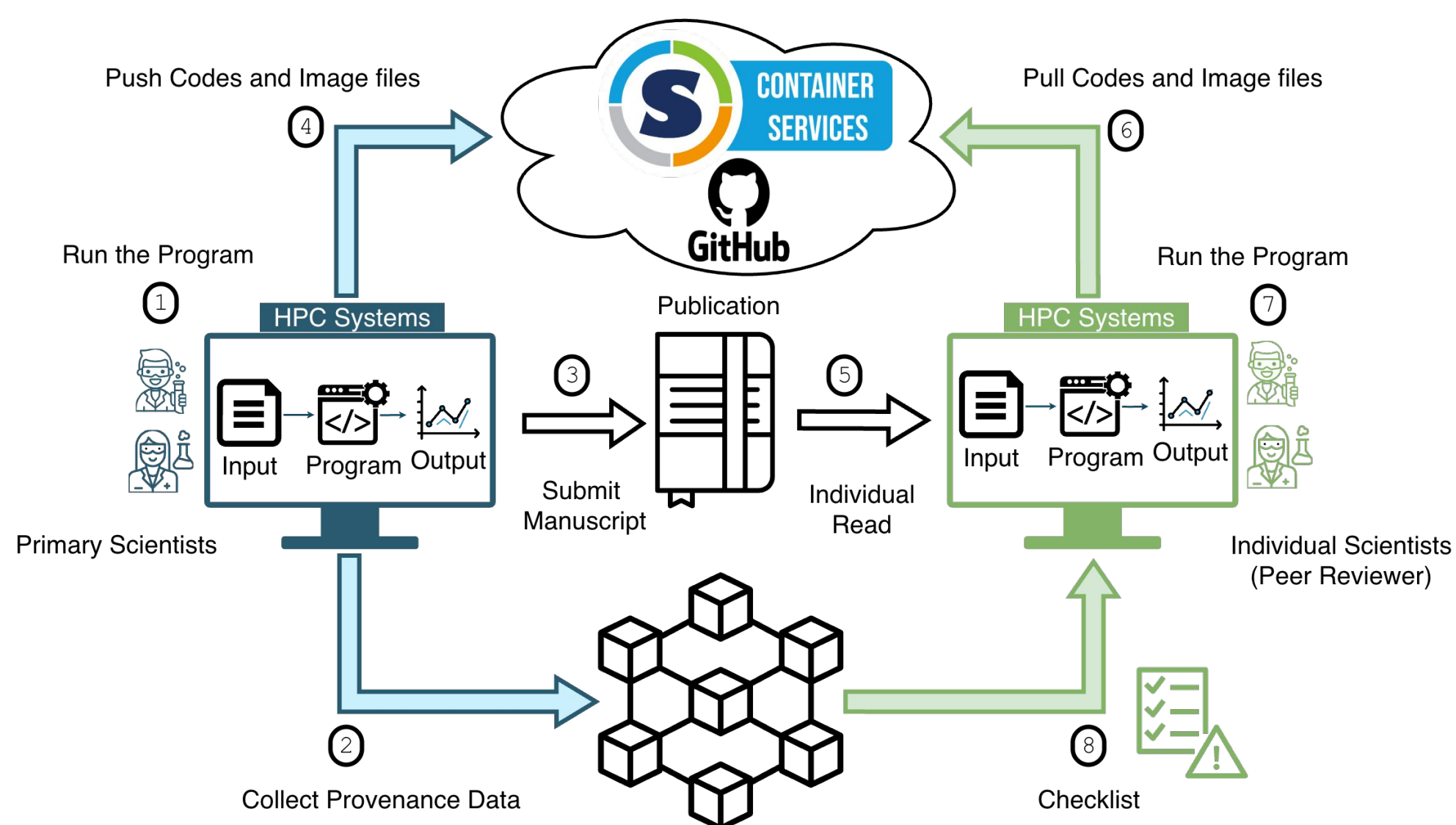


Fig1: Reproducibility Framework for Scientific Computing

Results

Add your information, graphs and images to this section.

Conclusion

Add your information, graphs and images to this section.

Recommendations

Add your information, graphs and images to this section.

Acknowledgements

Add your information, graphs and images to this section.

A Provenance-Aware Framework for Ensuring Reproducibility in High-Performance Computing

Pranav Chowdhury, Manjila Singh, Hari Teja Jajula, Trupesh Patel, Arpita Patel, Purushotham

The University of Alabama, Alabama Water Institute

Reproducibility Crisis in HPC

Fragmentation, erodes public trust
Complexity
Lack of documentation



How It Works

- Automates the process
- A Verifiable Execution Environment
 - System Configuration
 - Input Data
 - Execution Environment
 - Any other dependencies

Issues Addressed

Complexity: Hardware and software dependencies.

Provenance: Difficult to track. Verification is not easy.

Overhead: resource consumption to maintain provenance track.

A Provenance-Aware, Containerized Framework



Results

- A novel framework for key technologies
 - Containerization
 - Transferring

Push Center and

Publicly Available

Privacy Sciences

Colin

A Provenance-Aware Framework for Ensuring Reproducibility in High-Performance Computing

Md Kamal Hossain Chowdhury, Manjila Singh, Hari Teja Jajula, Trupesh Patel, Arpita Patel, Purushotham Bangalore
Department of Computer Science, The University of Alabama

The Reproducibility Crisis in HPC

1. The credibility of all scientific research relies on **reproducibility**.
2. In **High-Performance Computing (HPC)**, this challenge is magnified by complex software, massive datasets, and extreme parallelism.
3. **Lack of reproducibility** wastes resources, erodes public trust in scientific findings, and hinders collaboration.

Key Architectural Feature: Decoupled Design

To avoid burdening the HPC system, our framework uses a **two-stage approach**:

- **Stage 1 (On the HPC)**: Provenance data is collected efficiently by eBPF during the computation.
- **Stage 2 (Off the HPC)**: The collected data is then transferred via **gRPC** to a dedicated external server that handles all computationally intensive blockchain operations.
 - **Result**: The HPC resources remain focused on scientific tasks, not blockchain management.

A Provenance-Aware, Containerized Framework

We propose a novel framework that combines two key technologies:

1. **Containers (Docker/Singularity)**: To encapsulate the entire software environment, ensuring the application runs identically everywhere.
2. **Transparent Provenance Collection**: An automated system that tracks crucial metadata about the experiment without requiring user intervention.

How It Works: Key Features

- **Automated Metadata Collection**: Built on HPC I/O workloads to transparently and securely collect metadata on configuration, system details, and research data.
- **A Verifiable Workflow**: Our approach ensures every key component of an experiment is verifiable and traceable:
 - The system is built and run inside a **container**.
 - Input files are verified via **checksums**.
 - Execution **parameters are stored**.
 - Any changes to **output files are traced**.

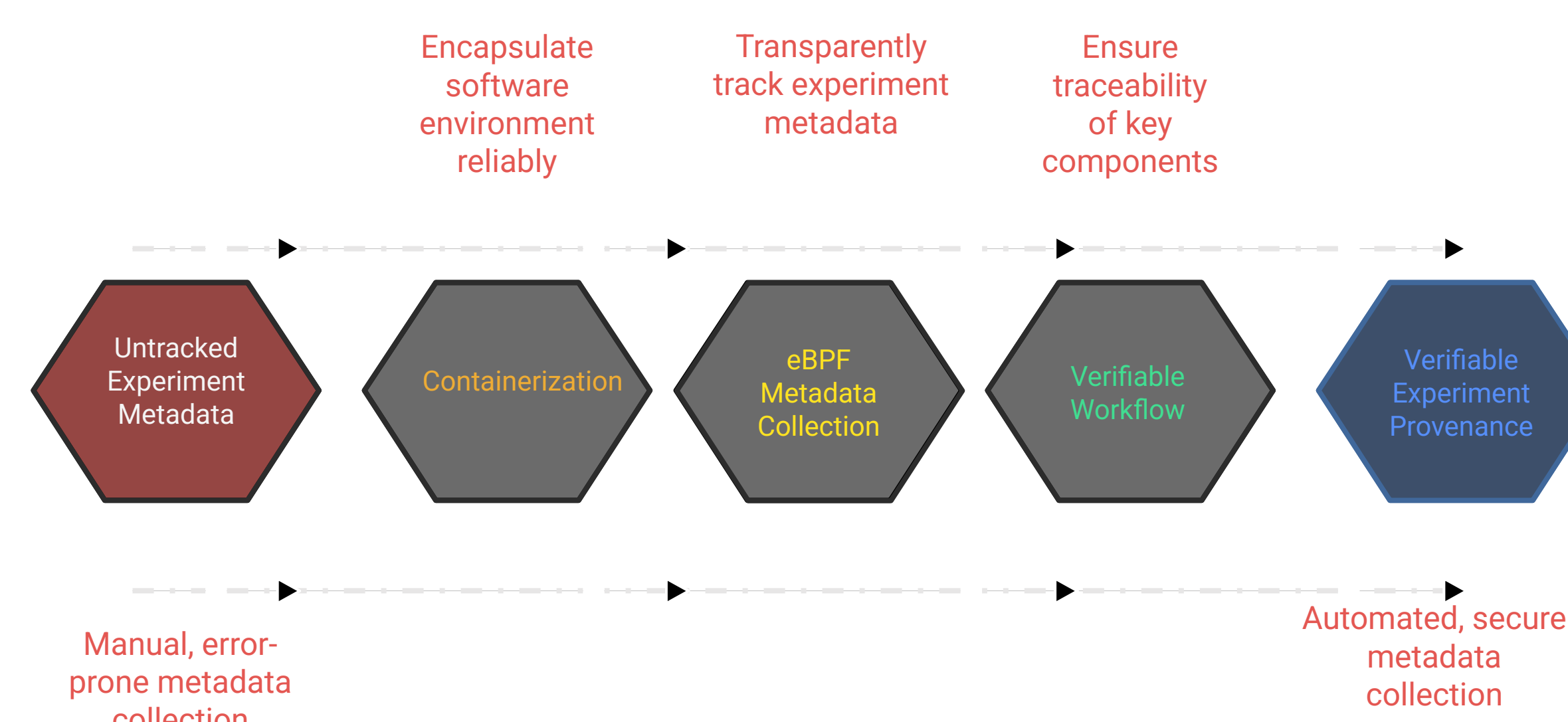


Fig1: Transparent Provenance Collection Framework

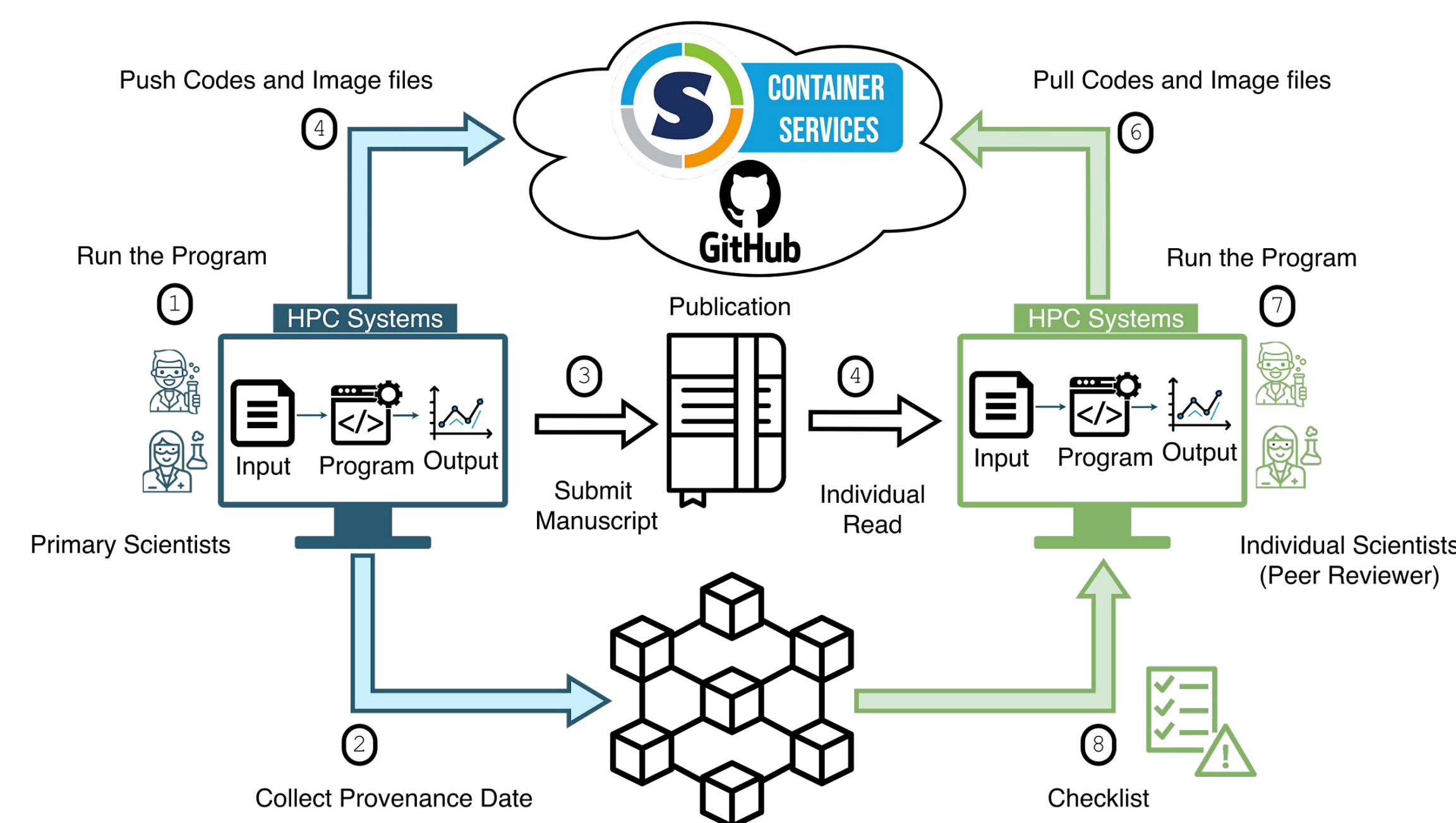


Fig2: Reproducibility Framework for Scientific Computing

Key Challenges Addressed

- **System Complexity**: Variations in configurations, libraries, and hardware make experiments difficult to replicate.
- **Lack of Provenance**: It is difficult to track the exact datasets, parameters, and software versions used to generate a result.
- **Data Integrity**: How can we verify that input data hasn't been corrupted or that the correct version was used?
- **Performance Overhead**: Traditional tracking methods are often too slow and resource-intensive for large-scale HPC files and workflows.

Workflow for Reproducibility

1. **Primary Researcher**:
 - Runs the experiment inside a Singularity container.
 - **eBPF** automatically traces the execution and sends the provenance data (metadata of inputs, outputs, image files, parameters) to the blockchain.
 - Publishes the paper, code (**GitHub**), and Singularity image (**Sylabs.io**).
2. **Peer Reviewer / Independent Scientist**:
 - Downloads the code and pulls the Singularity image from the public hubs.
 - Re-runs the exact experiment using the container.
 - Can **verify the results** and the computational steps against the immutable record stored on the blockchain to confirm repeatability and reproducibility.

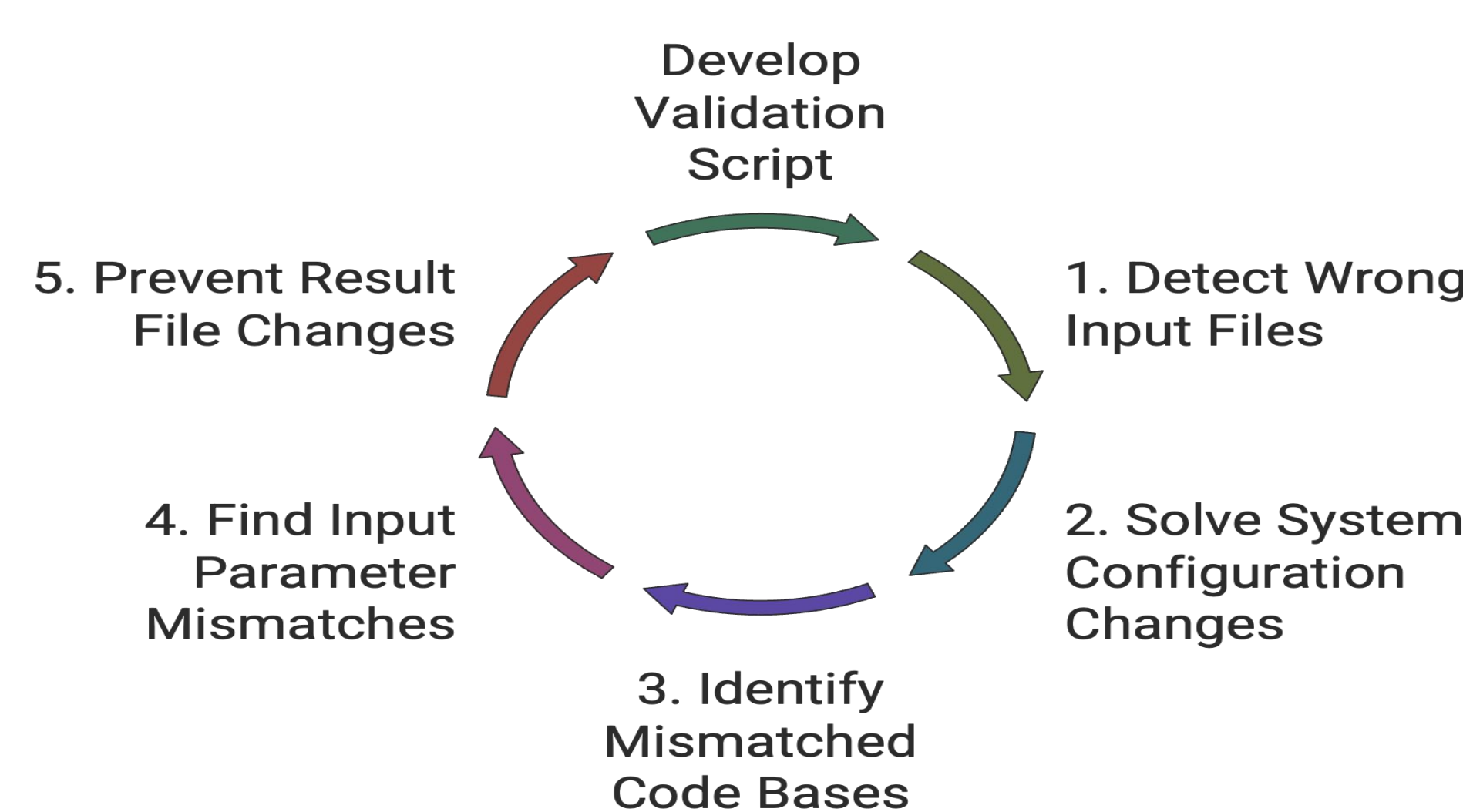


Fig3: Developed a script to check Reproducibility Validation

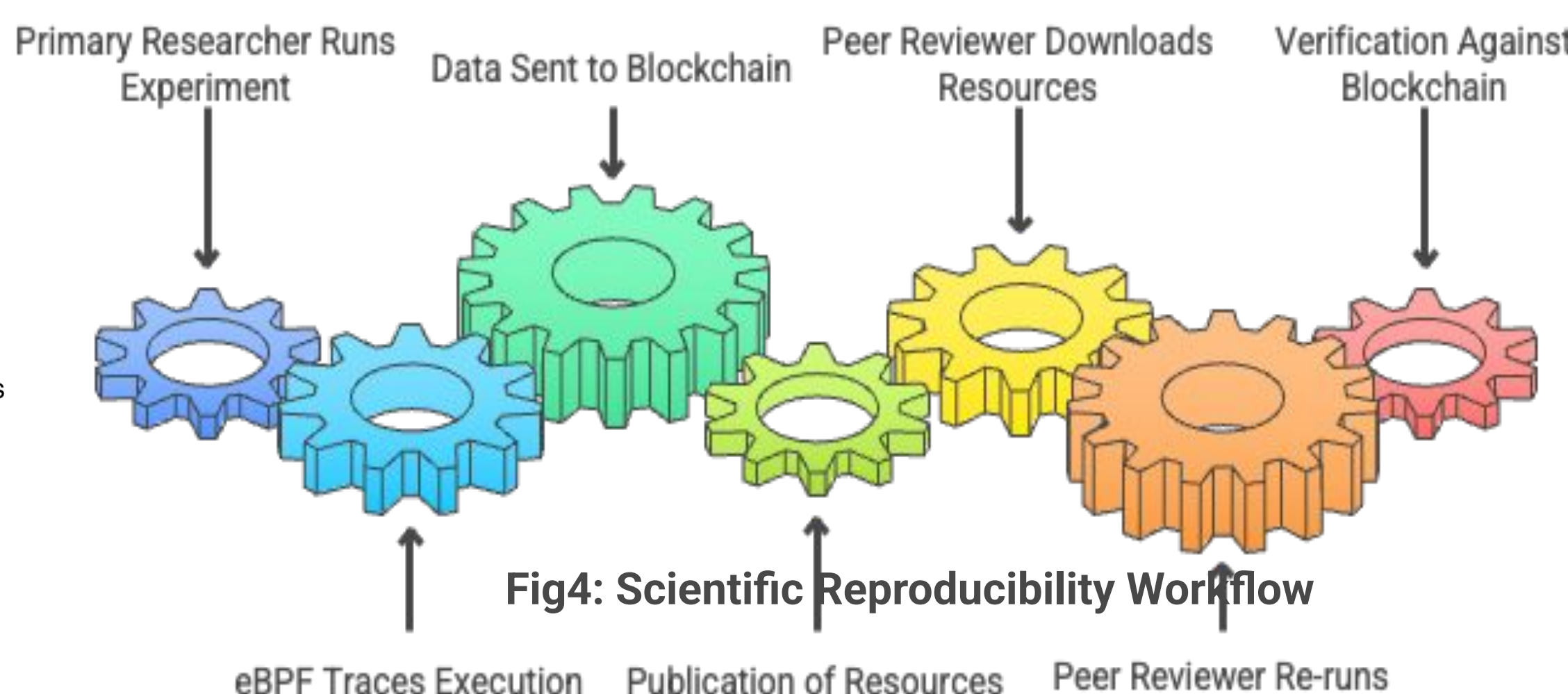


Fig4: Scientific Reproducibility Workflow

Key Finding: Minimal Performance Overhead

- Our framework was evaluated using real-world proxy applications on multiple HPC systems. Results show that our automated provenance collection adds **minimal overhead** to the overall application performance.
- This demonstrates our solution is not only effective but also **practical for everyday use** in demanding HPC environments.

Reproducibility check for NGEN In A BOX (NGIAB)

- NextGen In A Box (NGIAB): Next Generation Water Modeling Framework for Community Release (Singularity version).
- The project provides a user-friendly, containerized solution for running the NextGen National Water Resources Modeling Framework.
- Check Our framework in a real world application.
- Make containerization of the application and tested in different HPC Systems.
- This demonstrates our solution is reproducible and find the exact result for different runs.

MiniApps	Containerized Versions			
	Jetstream 2	ASC	UAHPC	Pantarhei
Lulesh	✓	✓	✓	✓
miniAMR	✓	✓	✓	✓
miniVite	✓	✓	✓	✓
NGIAB	✓	✓	✓	✓

Fig5: Reproducibility checks for different HPC systems

Provenance & Reproducibility

- **Blockchain-Based Provenance**: We use blockchain technology to store cryptographic provenance data for all input files and the **containerized image** files.
- **Traceable Executions**: Every single model execution is meticulously traced, with a record of the run stored on the blockchain.
- **Ensuring Reproducibility**: This immutable ledger allows secondary and third-party researchers to independently verify the **checksum** of input files and images. They can confirm that their files are identical to the originals, guaranteeing the integrity of the research.
- **Identical Outputs**: This robust system ensures that when the same inputs and container images are used, the experiments will generate **identical output files**, confirming the reproducibility of the scientific results.

Main Contributions & Impact

- **A Secure Framework**: We introduced a secure and reliable framework for data provenance on HPC systems.
- **A Practical Methodology**: We presented a methodology for conducting reproducible research that leverages provenance data.
- **Guidance for Researchers**: We provided guidance on how to design reproducible applications from the start.

Conclusions

Our work helps **increase the transparency, reliability, and trustworthiness** of computational science by making reproducibility in HPC practical and efficient.

Acknowledgements

References

1. <https://docs.ciroh.org/docs/services/external-re-sources/nsf-access/jetstream>
2. <https://github.com/CIROH-UA/NGIAB-HPCInfra>
3. <https://docs.ciroh.org/docs/services/on-prem/P-antarhei/>

Provenance-Aware Framework for Ensuring Reproducibility in High-Performance Computing

Sain Chowdhury, Manjila Singh, Hari Teja Jajula, Trupesh Patel, Arpita Patel, Purushotham

The University of Alabama, Alabama Water Institute

Reproducibility Crisis in HPC

Complexity, erodes public trust
Lack of transparency
Verification is not easy

Issues Addressed

Complexity: Hardware and software dependencies

Transparency: Difficult to track
Verification is not easy

Overhead: resource
Slow provenance track



How It Works

- Automated
- A Verifiable
- System level
- Input file
- Execution
- Any change



Results

- A novel framework for tracking provenance
- Two key components: Container-level and System-level
- Transparent

